Spring Boot Audit Log Service

Full Requirement Analysis & Solution Architecture Report

Java 17 • Spring Boot • Maven • Spring Data JPA / Hibernate ORM • JWT • JUnit 5 • Mockito • JaCoCo • OpenAPI • Logging • Monitoring

Source: Uploaded Interview Assignment — Build an AI-Assisted Software Engineering System — Audit Log Service, Version 2.0, dated 2026-08-03.

Purpose: Convert the supplied requirements into an implementable Spring Boot design and engineering plan, while separating explicit requirements from recommended technical choices.

Confidentiality: The source document is marked confidential and proprietary. Handle the source and this analysis according to its stated restrictions.

1. Executive Summary

The assignment requires a tamper-evident, append-only audit log service. The core workflow is: write events, query them, verify the hash chain, modify a record directly in the datastore to simulate tampering, and verify that the service detects the break. The assignment also adds retention/redaction and bulk export, plus an intentionally ambiguous compliance-reporting scenario.

The recommended implementation is a modular Spring Boot application using Java 17. Spring Data JPA with Hibernate ORM handles persistence; PostgreSQL is recommended for the relational datastore; Spring Security validates JWT bearer tokens and enforces roles; JUnit 5 and Mockito provide unit tests; JaCoCo supplies coverage; OpenAPI documents the REST contract; SLF4J/Logback provides structured logging; and Spring Boot Actuator with Micrometer provides health and metrics.

Important distinction: the assignment explicitly requires the audit behavior, hash chain, verification, retention/redaction/export scenarios, testing/documentation and quality validation. It does not mandate JWT, PostgreSQL, Maven, JUnit 5, Mockito, JaCoCo, OpenAPI, or a specific monitoring stack. Those are recommended implementation choices based on the requested technology baseline.

2. Requirement Traceability

The source explicitly defines event fields, append-only behavior, query filters/pagination, hash-chain integrity, verification, retention/redaction, export, and compliance clarification. □filecite□turn0file0□L73-L83□ □filecite□turn0file0□L115-L150□

Area	Source requirement	Proposed Spring Boot implementation	Type
Write API	Accept eventType, actorId, resourceType, resourceId, payload, timestamp; append-only.	POST /audit/events; server UTC timestamp recommended; validate and persist hashes.	Explicit + design
Query	Filter by actorId, resourceType/resourceId, eventType, from/to; pagination.	GET /audit/events using Pageable + criteria/specification.	Explicit
Hash chain	Own content hash + previous record hash/genesis value.	SHA-256 over deterministic canonical content + previousHash + chain sequence.	Explicit + design
Verification	GET /audit/verify reports intact/broken and first inconsistency.	Full ordered scan; validate content hash and previousHash linkage.	Explicit
Retention	Older records may be archived/soft-deleted; verification handles archives.	Configurable archive state and verification semantics.	Explicit + design
Redaction	Sensitive payload fields must be redactable without breaking chain.	Commitment/envelope approach with explicit post-redaction verification semantics.	Explicit + design
Export	Export by resourceId or actorId as verifiable bundle.	Export records + chain metadata + manifest; optional signature.	Explicit + design
Authentication	Not specified.	JWT bearer authentication via Spring Security.	Recommended
Authorization	Not specified.	AUDIT_WRITER / READER / EXPORTER / ADMIN authorities.	Recommended
Database	Not specified.	PostgreSQL + Spring Data JPA + Hibernate ORM.	Recommended
Quality	Quality gates, testing and validation are expected.	JUnit 5 + Mockito + JaCoCo + static analysis + dependency scan.	Recommended
Docs/Monitoring	API/schema definitions and production readiness are deliverables.	OpenAPI + Actuator/Micrometer + structured logging.	Recommended

3. Scope, Assumptions & Clarifications

The source requires three scenarios and explicitly asks the engineer to normalize ambiguity before implementation. filecite turn0file0 L141-L150

Recommended assumptions

- Timestamp: server-assigned UTC timestamp is authoritative. This prevents clients from controlling chain order.
- Ordering: use a monotonically increasing chainSequence. Do not rely on timestamps alone.
- Hash: SHA-256, with algorithm/version stored as metadata.
- Canonical content: deterministic encoding of eventType, actorId, resourceType, resourceId, payload and authoritative timestamp.
- Payload: PostgreSQL JSONB; canonicalization must be stable.
- Concurrency: serialize chain-head updates inside the same database transaction as event insertion.
- Security: JWT bearer authentication plus role/authority checks.
- Database integrity: application APIs expose no update/delete operation, but operational database controls are still required.

Scenario C clarification questions

- What exact access events must regulators see: successful, failed, privileged, service-to-service, or all?
- Which client-account resources are in scope?
- What retention period and reporting time zone apply?
- What evidence must a regulator independently verify?
- Who may generate reports and is report generation itself audited?
- How should archived and redacted records appear in reports?

4. Architecture

Recommended architecture: a modular monolith with clear API, security, application, domain, persistence and infrastructure boundaries. This is simpler to build, test and defend during a 2–3 day assessment than a distributed microservice design.

Client → JWT/Spring Security → REST Controller → Validation → Audit Service → Canonicalization/Hash Service → JPA Repository → PostgreSQL

Layer	Responsibilities	Technology
API	REST endpoints, DTOs, validation, error mapping.	Spring MVC, Bean Validation, OpenAPI
Security	JWT validation, authentication, authorization.	Spring Security
Application	Use cases, transactions, orchestration.	Spring services, @Transactional
Domain	Hash chain, canonicalization, verification rules.	Java 17
Persistence	Entities, repositories, dynamic filters.	Spring Data JPA, Hibernate
Infrastructure	DB migrations, logging, metrics, configuration.	PostgreSQL, Flyway, Logback, Actuator

Suggested packages

```
com.example.audit api/ controllers + DTOs application/ use cases/services domain/ event + hash-chain rules
persistence/ JPA entities + repositories security/ JWT + authorization infrastructure/ hashing, config, migrations
common/ exceptions, logging
```

5. Database & Spring Data JPA / ORM

PostgreSQL is recommended because transactions, JSONB, sequences and row-level locking are useful for an append-only chain. Spring Data JPA and Hibernate ORM provide the persistence abstraction.

Column	Type	Purpose	Index
id	BIGINT/UUID	Immutable record identifier.	PK
chain_sequence	BIGINT	Strict chain order.	Unique
event_type	VARCHAR	Event classification.	B-tree
actor_id	VARCHAR	Actor/service.	B-tree
resource_type	VARCHAR	Resource type.	Composite
resource_id	VARCHAR	Resource identifier.	Composite
payload	JSONB	Structured event details.	Optional GIN
event_timestamp	TIMESTAMPTZ	Authoritative UTC time.	B-tree
previous_hash	CHAR/VARCHAR	Hash of prior event or genesis.	No
content_hash	CHAR/VARCHAR	SHA-256 of canonical event content.	Optional unique
hash_algorithm	VARCHAR	Algorithm/version.	No
status	VARCHAR	ACTIVE/ARCHIVED/REDACTED semantics.	B-tree

JPA design rules

- Do not expose update/delete methods for audit events.
- Use JSONB mapping supported by the selected Hibernate version.
- Use Flyway for schema migrations.
- Use a dedicated chain_head row/table for serialized chain-head updates.
- Keep entity mutability tightly controlled; treat event fields as immutable after insert.

Canonical hash input

```
SHA-256( version | chainSequence | eventType | actorId | resourceType | resourceId | canonical(payload) |
eventTimestamp )
```

6. REST API Design

The source explicitly requires write, query and chain-verification APIs, and expects API/schema definitions in the deliverables. filecite turn0file0 L123-L127 filecite turn0file0 L167-L179

Method	Endpoint	Purpose	Authority
POST	/audit/events	Append event.	AUDIT_WRITER
GET	/audit/events	Query with filters + pagination.	AUDIT_READER
GET	/audit/verify	Verify full chain.	AUDIT_ADMIN
GET	/audit/events/{id}	Read one event (optional).	AUDIT_READER
GET	/audit/export	Export by actorId/resourceId.	AUDIT_EXPORTER
GET	/actuator/health	Health/readiness.	Restricted by deployment policy

Write request example

```
{ "eventType": "RECORD_UPDATED", "actorId": "user-123", "resourceType": "CLIENT_ACCOUNT", "resourceId": "acct-456",
"payload": {"field": "status", "newValue": "ACTIVE"} }
```

Query parameters: actorId, resourceType, resourceId, eventType, from, to, page, size, sort. Bound page size to protect the service.

Verification response should contain intact, checkedRecordCount, firstBrokenRecord/chainSequence, violationType and safe diagnostic details. Do not include sensitive payload data in errors.

7. Authentication & Authorization with JWT

JWT is a recommended security choice rather than a source-mandated requirement. Spring Security should validate signature, issuer, audience, expiration and not-before claims before the request reaches business logic.

Authentication flow

1. Client obtains a signed JWT from an identity provider or controlled demo issuer.
2. Client sends Authorization: Bearer .
3. Spring Security validates the token and creates the authenticated principal.
4. Endpoint authorization checks required authorities/scopes.
5. Business logic receives only an authenticated, authorized request.

Authority	Access
AUDIT_WRITER	Append events.
AUDIT_READER	Query events and view verification results as permitted.
AUDIT_EXPORTER	Generate verifiable exports.
AUDIT_ADMIN	Verification/admin diagnostics.

Security controls

- Prefer asymmetric signing such as RS256/ES256 where appropriate.
- Validate issuer, audience, expiry and signature.
- Never log JWTs, Authorization headers, passwords or secrets.
- Use TLS in deployed environments.
- Protect actuator and Swagger endpoints according to environment policy.
- Return generic 401/403 responses without leaking security internals.

8. Hash Chain & Tamper-Evidence

The source requires every record to contain a hash of its own content and the immediately preceding record's hash, with a genesis value for the first record. Verification must identify the first inconsistency when the chain is broken. filecite turn0file0 L118-L127

Write algorithm

1. Start a database transaction.
2. Lock/read the current chain head.
3. Allocate the next chainSequence.
4. Create deterministic canonical content.
5. Compute SHA-256 contentHash.
6. Set previousHash to the current chain head or genesis value.
7. Insert the immutable event and atomically update the chain head.
8. Commit the transaction.

Verification algorithm

1. Read records in chainSequence order.
2. Validate genesis previousHash on the first record.
3. Recompute each contentHash.
4. Compare each previousHash with the prior record's contentHash.
5. Stop at the first mismatch and classify the violation.
6. Return intact=true only when all checks pass.

Concurrency control

A dedicated chain_head table is recommended. Lock its single active row using the database's row-locking mechanism inside the same transaction as event insertion. This prevents two concurrent writers from using the same previousHash.

Operational limitation

Hash chaining detects inconsistency; it does not independently prove who modified a database row. Production controls should therefore include least-privilege DB accounts, restricted administrator access, database auditing, backups and monitoring.

9. Scenario B — Retention, Redaction & Bulk Export

Retention

The source allows archiving or soft deletion for records older than a configurable window and requires verification to handle legitimate archival without reporting a false break. filecite turn0file0 L128-L137

Recommended model: retain chain metadata and mark records ARCHIVED, or move them to a controlled archive that preserves their chain metadata. Verification must distinguish a legitimate archival boundary from an integrity failure.

Redaction

Simply replacing sensitive data changes the original hash input. A practical design is a commitment/envelope model: preserve an immutable cryptographic commitment to the original value while replacing the readable value with a redaction marker and recording controlled redaction metadata. Verification then proves the committed/redacted state without requiring the original plaintext.

Bulk export

- Filter by resourceId or actorId.
- Include event data plus id, chainSequence, previousHash, contentHash, algorithm and chain/genesis metadata.
- Include adjacent-chain evidence or sufficient metadata for the recipient to independently verify the exported subset.
- Optionally sign the export manifest when stronger provenance is needed.
- Apply the same authorization and data-minimization rules as normal reads.

10. Logging, Error Handling & Monitoring

Use SLF4J/Logback with structured logs. Logging must help operational troubleshooting and security investigations without leaking sensitive data.

Level	Use	Rule
ERROR	Unexpected DB/application failures.	Safe IDs + correlation ID; no payload/JWT.
WARN	Rejected access, validation anomalies, verification failure.	Avoid repetitive noise.
INFO	Startup, high-level operations, verification result.	Structured, non-sensitive fields.
DEBUG	Development diagnostics.	Restrict in production.

Recommended MDC fields: traceId, requestId, endpoint, method, status, durationMs, and safe actor identifier where permitted. Never log Authorization headers, JWTs, secrets or sensitive payload content.

Monitoring stack

- Spring Boot Actuator for health/readiness and operational endpoints.
- Micrometer for application metrics.
- Prometheus-compatible registry recommended for production metrics.
- Optional OpenTelemetry/Micrometer tracing for distributed environments.

Useful metrics: write count, query count, write/query latency, verification duration, verification failures, export count, redaction count, DB errors and authorization failures.

11. Unit Testing with JUnit 5 & Mockito

The source requires production-quality code, unit/integration tests and validation. filecite turn0file0 L67-L72

Test level	Tools	Focus
Domain	JUnit 5	Canonicalization, hashing, verification rules.
Service	JUnit 5 + Mockito	Append workflow, failures, repository interactions.
Repository	Spring Boot Test / JPA	Queries, mappings, JSONB and indexes.
API	MockMvc + Security Test	HTTP contract, validation, JWT authorities.
E2E	Testcontainers recommended	Real PostgreSQL transactions and tamper simulation.

Critical tests

- Genesis event has the correct genesis previousHash.
- Second event references the first contentHash.
- Changing event fields directly in DB is detected.
- Changing contentHash is detected.
- Changing previousHash is detected.
- Middle-record corruption reports the first inconsistency.
- Concurrent writes preserve sequence and linkage.
- Required filters and pagination are deterministic.
- Archive/redaction semantics do not create false positives.
- JWT authentication/authorization paths are tested.

12. Maven, JaCoCo & Code Quality

Maven should provide a single reproducible build command such as mvn clean verify. Quality gates should execute as part of the normal CI pipeline.

Recommended pipeline gates

- Compile with Java 17.
- Run unit and integration tests.
- Generate JaCoCo coverage.
- Run static analysis such as Checkstyle/SpotBugs and/or SonarQube.
- Run dependency vulnerability scanning.
- Validate/generate OpenAPI documentation.
- Fail CI for agreed critical quality/security findings.

Metric	Recommended target	Reason
Line coverage	≥ 80% overall	Practical baseline.
Branch coverage	≥ 70% overall	Useful for security/error logic.
Hash/verification domain	≥ 90%	Highest correctness risk.
Quality defects	No blocker/critical	Prevent regressions.

Coverage numbers are recommendations, not source requirements; align them with the target organization's standards if known.

JaCoCo Maven concept

```
jacoco:prepare-agent test / integration-test jacoco:report jacoco:check verify
```

13. API Documentation

Use OpenAPI 3, preferably through springdoc-openapi, to document request/response schemas, validation constraints, security scheme, roles/scopes, pagination, error responses and verification semantics.

Swagger UI may be enabled for development. In production it should be protected or disabled according to the deployment security policy.

14. Recommended Technology Stack

Capability	Choice	Purpose
Language	Java 17	Requested baseline.
Framework	Spring Boot 3.x compatible with Java 17	REST, DI, security, actuator.
Build	Maven	Requested build tool.
DB	PostgreSQL	Transactions, JSONB, locking.
ORM	Spring Data JPA + Hibernate	Requested persistence approach.
Security	Spring Security + JWT resource server	JWT auth + authorization.
Testing	JUnit 5 + Mockito	Requested unit testing stack.
Coverage	JaCoCo	Coverage report/gate.
Docs	OpenAPI 3 / springdoc	API contract and UI.
Logging	SLF4J + Logback	Structured application logs.
Monitoring	Actuator + Micrometer	Health + metrics.
Migration	Flyway	Versioned DB schema.
Integration test	Testcontainers	Real PostgreSQL testing.

15. Implementation Plan & Task Decomposition

The source explicitly evaluates requirement understanding, decomposition, AI-assisted execution, quality gates and engineer ownership. filecite turn0file0 L55-L70

Phase	Tasks	Acceptance criteria
1. Foundation	Java 17, Maven, Spring Boot, profiles, Flyway, DB.	App starts and migration succeeds.
2. Domain	Event model, canonical JSON, hash service, verification.	Deterministic hash tests pass.
3. Persistence	JPA entities, repositories, indexes, chain head.	Writes are transactional and ordered.
4. API	POST, GET, verify, validation and error mapping.	API/integration tests pass.
5. Security	JWT resource server and role policies.	Authorized/unauthorized tests pass.
6. Scenario B	Retention, redaction and export.	Behavior matches documented semantics.
7. Quality	JUnit, Mockito, JaCoCo, static analysis, dependency scan.	mvn verify is green.
8. Observability	Logging, Actuator, metrics.	Operational signals are available safely.
9. Documentation	Architecture, setup, assumptions, trade-offs, AI usage log.	Reviewer can run and defend solution.

Recommended order

1. Normalize requirements and write acceptance criteria.
2. Implement and test hash-chain domain logic first.
3. Implement persistence and chain-head concurrency control.
4. Add REST API and validation.
5. Add JWT security.
6. Demonstrate tamper detection.
7. Implement Scenario B and define Scenario C scope.
8. Add quality gates, API docs and monitoring.
9. Run final validation and prepare the engineering summary.

16. Risks, Trade-offs & Validation

Risk	Impact	Mitigation
Concurrent writers	Broken chain linkage.	Transactional chain-head lock + concurrency tests.
Non-deterministic JSON	False tamper detection.	Canonical JSON serialization.
Direct DB edits	Integrity failure detected; root cause not automatically known.	Verification + DB controls + operational audit.
Redaction	Changing values can invalidate hashes.	Commitment/envelope strategy.
Large chain verification	High latency/resource use.	Batch/stream reads, metrics and archive policy.
Sensitive logs	Data leakage.	Allow-list logging and secret scanning.
JWT configuration	Unauthorized access.	Issuer/audience/signature/expiry validation + tests.

Validation matrix

- Functional: create → query → verify → tamper DB → verify again.
- Security: valid/invalid/expired JWT and role matrix.
- Persistence: migrations, queries, pagination, JSONB.
- Integrity: first/middle/last record corruption.
- Concurrency: parallel writers.
- Performance: large queries and full-chain verification baseline.
- Quality: tests, JaCoCo, static analysis, dependency scan.
- Operations: health, metrics, safe logs and failure behavior.

17. Final Engineering Checklist

- Requirements, ambiguities and assumptions documented.
- Architecture, data model and API contract documented.
- Java 17 + Spring Boot + Maven build is reproducible.
- PostgreSQL schema and Flyway migrations are versioned.
- Spring Data JPA / Hibernate mappings are tested.
- No update/delete audit APIs are exposed.
- Query API supports all required filters and pagination.
- Hash chain uses deterministic canonicalization and SHA-256.
- Verification identifies the first inconsistency.
- Direct database tamper test demonstrates detection.
- Retention/archive semantics are documented and tested.
- Redaction design preserves documented integrity/privacy properties.
- Bulk export contains enough metadata for verification.
- JWT authentication and role authorization are tested.
- Sensitive values are excluded from logs/errors.
- JUnit 5 + Mockito unit tests cover core logic.
- Integration tests cover DB and security boundaries.
- JaCoCo report and build gate are configured.
- Static quality and dependency security checks are configured.
- OpenAPI documentation covers APIs and schemas.
- Actuator/Micrometer monitoring is protected and useful.
- Setup, limitations, trade-offs and assumptions are documented.
- AI usage traceability is maintained as required by the assignment.
- ATTESTATION.md is included with the required statement.

18. Conclusion

The recommended solution is a secure, testable Spring Boot modular monolith centered on an immutable audit event model and a deterministic cryptographic hash chain. The key engineering challenge is preserving integrity under concurrency, detecting direct database tampering, and defining retention/redaction semantics without weakening the evidence model. JWT, JPA/Hibernate, JUnit 5/Mockito, JaCoCo, OpenAPI, structured logging and Actuator/Micrometer provide the requested production-oriented foundation.

> The source emphasizes engineer-owned judgment rather than checklist compliance. During review, present each non-mandated technology as a deliberate trade-off, and be prepared to explain assumptions, alternatives, risks and limitations.

Source basis: Uploaded 4-page Charles Schwab & Co., Inc. assignment. The source states that it is confidential and proprietary. filecite turn0file0 L1-L8